

GEBZE TECHNICAL UNIVERSITY

CSE513 Assignment 1
CPU Scheduling Simulator

https://cereenyildiirim.github.io/osassignment/

Model and local setup

Scheduling model

The simulator compares FCFS, non-preemptive SJF, SRTF and Round Robin on one CPU. Each process has one CPU burst, with no I/O and zero context-switch cost. This is a policy model, not Ubuntu's actual scheduler.

FCFS follows arrival order. SJF selects the shortest ready burst when the CPU becomes free. SRTF preempts only for a strictly shorter remaining time. RR uses a FIFO queue, adding boundary arrivals before requeuing an unfinished process. Other ties use arrival time, then input order. The CPU is idle only when no process is ready.

Metrics

Turnaround (TAT) = completion - arrival
Waiting (WT) = turnaround - CPU burst
Response (RT) = first start - arrival

Tables use A for arrival, B for burst, S for first start and C for completion. Intervals use [start, end), so 2-5 represents three time units.

Run locally

Download the source ZIP from the repository or clone the main branch. Open a terminal in the project folder and run:

python3 -m http.server 8000 --bind 127.0.0.1

Open **http://127.0.0.1:8000** in a modern browser. On Windows, **py -m http.server 8000 --bind 127.0.0.1** can be used instead. Stop the server with Ctrl+C. Python 3 is needed only to serve the files locally; no npm installation, backend or database is required. JavaScript modules should be opened through the server rather than by double-clicking index.html.

Load Workload A or B, choose an algorithm, set the quantum for RR and click **Run simulation**. Processes can also be added, edited or removed. Node.js 22 or later is needed only for the tests: **node --test scheduler.test.mjs**. All 16 tests pass.

The same local setup instructions, dependencies and limitations are documented in **README.md**, included with the source code.

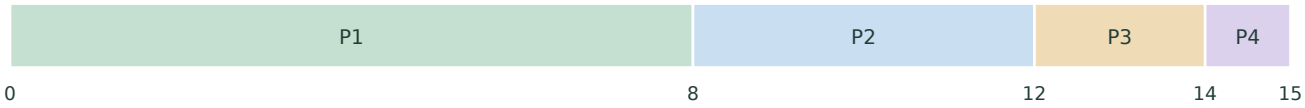
Source code: github.com/cereenyildiirim/osassignment/tree/main

Workload A: FCFS

Input: P1 (arrival 0, burst 8); P2 (arrival 1, burst 4); P3 (arrival 2, burst 2); P4 (arrival 4, burst 1).

Hand calculation

P1 is the only ready process at time 0, so it runs to completion at time 8. P2, P3 and P4 then run in arrival order. P4 needs only one time unit but waits behind the longer jobs.



0-8: P1 | 8-12: P2 | 12-14: P3 | 14-15: P4

Process	A	B	S	C	TAT	WT	RT
P1	0	8	0	8	8	0	0
P2	1	4	8	12	11	7	7
P3	2	2	12	14	12	10	10
P4	4	1	14	15	11	10	10

Hand-calculated averages: TAT = **10.5**, WT = **6.75**, RT = **6.75**.

For P2: TAT = 12 - 1 = 11; WT = 11 - 4 = 7; RT = 8 - 1 = 7.

Simulator output

Process	A	B	S	C	TAT	WT	RT
P1	0	8	0	8	8	0	0
P2	1	4	8	12	11	7	7
P3	2	2	12	14	12	10	10
P4	4	1	14	15	11	10	10

Output averages: TAT = 10.5, WT = 6.75, RT = 6.75.

Comparison

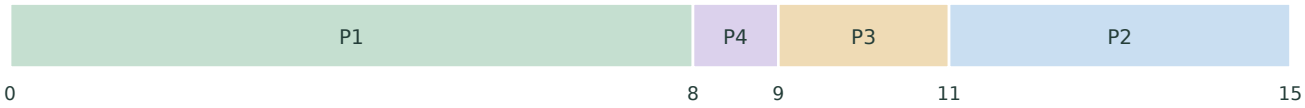
Every execution interval, per-process metric and average matches the hand calculation. No discrepancy was found. The browser check also compares the displayed table and interval list with the engine output for this case.

Workload A: SJF

Input: P1 (arrival 0, burst 8); P2 (arrival 1, burst 4); P3 (arrival 2, burst 2); P4 (arrival 4, burst 1).

Hand calculation

P1 still runs first: the shorter jobs have not arrived at time 0, and SJF does not preempt. At time 8, all three waiting jobs are ready. Their burst lengths give the order P4, P3, P2.



0-8: P1 | 8-9: P4 | 9-11: P3 | 11-15: P2

Process	A	B	S	C	TAT	WT	RT
P1	0	8	0	8	8	0	0
P2	1	4	11	15	14	10	10
P3	2	2	9	11	9	7	7
P4	4	1	8	9	5	4	4

Hand-calculated averages: TAT = **9**, WT = **5.25**, RT = **5.25**.

For P2: TAT = 15 - 1 = 14; WT = 14 - 4 = 10; RT = 11 - 1 = 10.

Simulator output

Process	A	B	S	C	TAT	WT	RT
P1	0	8	0	8	8	0	0
P2	1	4	11	15	14	10	10
P3	2	2	9	11	9	7	7
P4	4	1	8	9	5	4	4

Output averages: TAT = 9, WT = 5.25, RT = 5.25.

Comparison

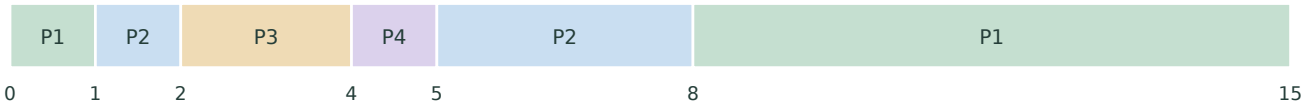
Every execution interval, per-process metric and average matches the hand calculation. No discrepancy was found. The browser check also compares the displayed table and interval list with the engine output for this case.

Workload A: SRTF

Input: P1 (arrival 0, burst 8); P2 (arrival 1, burst 4); P3 (arrival 2, burst 2); P4 (arrival 4, burst 1).

Hand calculation

At time 1, P1 has 7 units left and P2 needs 4, so P2 preempts P1. At time 2, P2 has 3 units left and P3 needs 2, so P3 takes over. P3 finishes at time 4, when P4 arrives. P4 runs next, followed by the remaining parts of P2 and P1.



0-1: P1 | 1-2: P2 | 2-4: P3 | 4-5: P4 | 5-8: P2 | 8-15: P1

Process	A	B	S	C	TAT	WT	RT
P1	0	8	0	15	15	7	0
P2	1	4	1	8	7	3	0
P3	2	2	2	4	2	0	0
P4	4	1	4	5	1	0	0

Hand-calculated averages: TAT = **6.25**, WT = **2.5**, RT = **0**.

For P2: TAT = 8 - 1 = 7; WT = 7 - 4 = 3; RT = 1 - 1 = 0.

Simulator output

Process	A	B	S	C	TAT	WT	RT
P1	0	8	0	15	15	7	0
P2	1	4	1	8	7	3	0
P3	2	2	2	4	2	0	0
P4	4	1	4	5	1	0	0

Output averages: TAT = 6.25, WT = 2.5, RT = 0.

Comparison

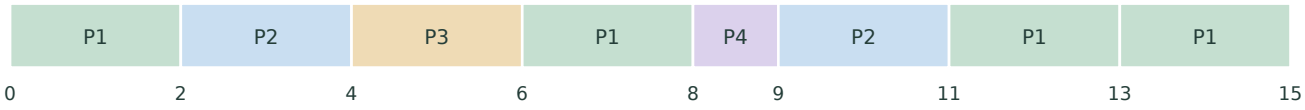
Every execution interval, per-process metric and average matches the hand calculation. No discrepancy was found. The browser check also compares the displayed table and interval list with the engine output for this case.

Workload A: RR (q = 2)

Input: P1 (arrival 0, burst 8); P2 (arrival 1, burst 4); P3 (arrival 2, burst 2); P4 (arrival 4, burst 1).

Hand calculation

With q=2, the queue at time 2 is P2, P3, P1. At time 4, P4 arrives before the unfinished P2 is requeued, giving P3, P1, P4, P2. P4 finishes after one unit. The final two P1 slices are kept separate so the quantum boundary at time 13 remains visible.



0-2: P1 | 2-4: P2 | 4-6: P3 | 6-8: P1 | 8-9: P4 | 9-11: P2 | 11-13: P1 | 13-15: P1

Process	A	B	S	C	TAT	WT	RT
P1	0	8	0	15	15	7	0
P2	1	4	2	11	10	6	1
P3	2	2	4	6	4	2	2
P4	4	1	8	9	5	4	4

Hand-calculated averages: TAT = **8.5**, WT = **4.75**, RT = **1.75**.

For P2: TAT = 11 - 1 = 10; WT = 10 - 4 = 6; RT = 2 - 1 = 1.

Simulator output

Process	A	B	S	C	TAT	WT	RT
P1	0	8	0	15	15	7	0
P2	1	4	2	11	10	6	1
P3	2	2	4	6	4	2	2
P4	4	1	8	9	5	4	4

Output averages: TAT = 8.5, WT = 4.75, RT = 1.75.

Comparison

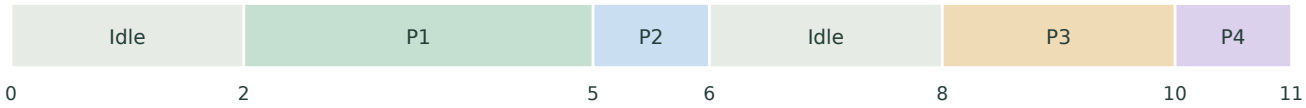
Every execution interval, per-process metric and average matches the hand calculation. No discrepancy was found. The browser check also compares the displayed table and interval list with the engine output for this case.

Workload B: FCFS

Input: P1 (arrival 2, burst 3); P2 (arrival 2, burst 1); P3 (arrival 8, burst 2); P4 (arrival 9, burst 1).

Hand calculation

No process is ready during 0-2 or 6-8, so both intervals are idle. P1 and P2 arrive together at time 2; input order puts P1 first. P4 arrives while P3 is running but cannot preempt it.



0-2: Idle | 2-5: P1 | 5-6: P2 | 6-8: Idle | 8-10: P3 | 10-11: P4

Process	A	B	S	C	TAT	WT	RT
P1	2	3	2	5	3	0	0
P2	2	1	5	6	4	3	3
P3	8	2	8	10	2	0	0
P4	9	1	10	11	2	1	1

Hand-calculated averages: TAT = **2.75**, WT = **1**, RT = **1**.

For P2: TAT = 6 - 2 = 4; WT = 4 - 1 = 3; RT = 5 - 2 = 3.

Simulator output

Process	A	B	S	C	TAT	WT	RT
P1	2	3	2	5	3	0	0
P2	2	1	5	6	4	3	3
P3	8	2	8	10	2	0	0
P4	9	1	10	11	2	1	1

Output averages: TAT = 2.75, WT = 1, RT = 1.

Comparison

Every execution interval, per-process metric and average matches the hand calculation. No discrepancy was found. The browser check also compares the displayed table and interval list with the engine output for this case.

Workload B: SJF

Input: P1 (arrival 2, burst 3); P2 (arrival 2, burst 1); P3 (arrival 8, burst 2); P4 (arrival 9, burst 1).

Hand calculation

At time 2, P2 is shorter than P1 and runs first. P1 follows. After the idle interval, P3 starts at time 8 and finishes before P4 can run because SJF is non-preemptive.



0-2: Idle | 2-3: P2 | 3-6: P1 | 6-8: Idle | 8-10: P3 | 10-11: P4

Process	A	B	S	C	TAT	WT	RT
P1	2	3	3	6	4	1	1
P2	2	1	2	3	1	0	0
P3	8	2	8	10	2	0	0
P4	9	1	10	11	2	1	1

Hand-calculated averages: TAT = **2.25**, WT = **0.5**, RT = **0.5**.

For P2: TAT = 3 - 2 = 1; WT = 1 - 1 = 0; RT = 2 - 2 = 0.

Simulator output

Process	A	B	S	C	TAT	WT	RT
P1	2	3	3	6	4	1	1
P2	2	1	2	3	1	0	0
P3	8	2	8	10	2	0	0
P4	9	1	10	11	2	1	1

Output averages: TAT = 2.25, WT = 0.5, RT = 0.5.

Comparison

Every execution interval, per-process metric and average matches the hand calculation. No discrepancy was found. The browser check also compares the displayed table and interval list with the engine output for this case.

Workload B: SRTF

Input: P1 (arrival 2, burst 3); P2 (arrival 2, burst 1); P3 (arrival 8, burst 2); P4 (arrival 9, burst 1).

Hand calculation

The early decisions match SJF. At time 9, P3 has one unit left and the new P4 also needs one unit. Equal remaining time does not cause preemption, so P3 continues. The whole schedule therefore matches SJF.



0-2: Idle | 2-3: P2 | 3-6: P1 | 6-8: Idle | 8-10: P3 | 10-11: P4

Process	A	B	S	C	TAT	WT	RT
P1	2	3	3	6	4	1	1
P2	2	1	2	3	1	0	0
P3	8	2	8	10	2	0	0
P4	9	1	10	11	2	1	1

Hand-calculated averages: TAT = **2.25**, WT = **0.5**, RT = **0.5**.

For P2: TAT = 3 - 2 = 1; WT = 1 - 1 = 0; RT = 2 - 2 = 0.

Simulator output

Process	A	B	S	C	TAT	WT	RT
P1	2	3	3	6	4	1	1
P2	2	1	2	3	1	0	0
P3	8	2	8	10	2	0	0
P4	9	1	10	11	2	1	1

Output averages: TAT = 2.25, WT = 0.5, RT = 0.5.

Comparison

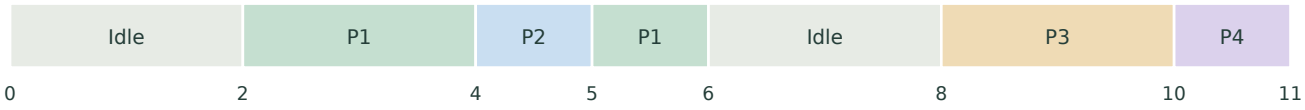
Every execution interval, per-process metric and average matches the hand calculation. No discrepancy was found. The browser check also compares the displayed table and interval list with the engine output for this case.

Workload B: RR (q = 2)

Input: P1 (arrival 2, burst 3); P2 (arrival 2, burst 1); P3 (arrival 8, burst 2); P4 (arrival 9, burst 1).

Hand calculation

P1 and P2 enter the queue in input order. P1 runs for two units, leaving one, then moves behind P2. P2 finishes at time 5 and P1 at time 6. Later, P4 joins the queue at time 9 and waits for P3 to finish its slice at time 10.



0-2: Idle | 2-4: P1 | 4-5: P2 | 5-6: P1 | 6-8: Idle | 8-10: P3 | 10-11: P4

Process	A	B	S	C	TAT	WT	RT
P1	2	3	2	6	4	1	0
P2	2	1	4	5	3	2	2
P3	8	2	8	10	2	0	0
P4	9	1	10	11	2	1	1

Hand-calculated averages: TAT = **2.75**, WT = **1**, RT = **0.75**.

For P2: TAT = 5 - 2 = 3; WT = 3 - 1 = 2; RT = 4 - 2 = 2.

Simulator output

Process	A	B	S	C	TAT	WT	RT
P1	2	3	2	6	4	1	0
P2	2	1	4	5	3	2	2
P3	8	2	8	10	2	0	0
P4	9	1	10	11	2	1	1

Output averages: TAT = 2.75, WT = 1, RT = 0.75.

Comparison

Every execution interval, per-process metric and average matches the hand calculation. No discrepancy was found. The browser check also compares the displayed table and interval list with the engine output for this case.

Additional tests

1. Idle CPU

Input: A(arrival 2, burst 1), B(arrival 5, burst 1). Expected under all four algorithms: 0-2 Idle | 2-3 A | 3-5 Idle | 5-6 B. Both processes have TAT=1 and WT=RT=0. This checks that the initial gap and the gap between jobs appear in the timeline. Result: passed.

2. Simultaneous arrivals and a tie

Input order: Z(arrival 0, burst 1), A(arrival 0, burst 1). Expected under all four algorithms: 0-1 Z | 1-2 A. Z has TAT=1 and WT=RT=0; A has TAT=2 and WT=RT=1. This catches implementations that sort tied processes by their IDs instead of their input positions. Result: passed.

3. SRTF preemption

Input: A(arrival 0, burst 5), B(arrival 1, burst 1). Expected SRTF timeline: 0-1 A | 1-2 B | 2-6 A. A has TAT=6, WT=1, RT=0; B has TAT=1 and WT=RT=0. This checks preemption, preservation of remaining time and resumption of the longer job. Result: passed.

General correctness properties

Property	Check
Exact CPU burst	Add the durations of all slices for each process.
No early execution	Each slice starts at or after the process arrival.
No overlap	Each interval ends where the next interval starts. Idle gaps are explicit.
Nonnegative WT and RT	Check both metrics in every result row.
No avoidable idle time	No arrived, unfinished process exists during an idle interval.

These properties are checked for the eight assignment examples and the three additional workloads. Validation tests also reject empty lists, duplicate or blank IDs, invalid times and invalid quantum values. Run the suite with **node --test scheduler.test.mjs**.

Analysis: results and quantum size

1. Which algorithm performs best for Workload A?

Algorithm	Average TAT	Average WT	Average RT
FCFS	10.5	6.75	6.75
SJF	9	5.25	5.25
SRTF	6.25	2.5	0
RR	8.5	4.75	1.75

SRTF gives the lowest average waiting time, 2.5, and the lowest average response time, 0. P2 preempts P1, then P3 preempts P2. P4 arrives just as P3 finishes. Each process therefore starts at its arrival time in this workload. That explains the zero response average; it is not a general guarantee of SRTF.

2. What changes with RR quantum 1, 2 and 4?

The following results use Workload A with three quantum values.

Workload A

Quantum	Average TAT	Average WT	Average RT
1	8.5	4.75	0.75
2	8.5	4.75	1.75
4	9.25	5.5	3.75

For A, reducing q from 2 to 1 improves response from 1.75 to 0.75 but leaves waiting at 4.75 and turnaround at 8.5. A shorter quantum does not improve every metric. It gets processes their first CPU time sooner, but also spreads their work across more turns. Completion time depends on the resulting queue order, not just how soon a process first runs.

These results assume zero switching cost. In a real system, extra context switches would add another cost to very small quantum values.

Analysis: assumptions and verification

3. When do SJF and SRTF produce the same schedule?

They agree when no arrival would force SRTF to interrupt the process selected by SJF. In particular, no newly arriving job may be strictly shorter than the running job's remaining time. If every job is ready at the start, this condition is satisfied. Workload B is another example: P4 arrives when P3 has one unit left, so the equal-time rule lets P3 finish.

4. Why estimate CPU bursts in a real operating system?

A real operating system usually does not know when a program will finish its next CPU burst or request I/O. That depends on the program and its input. A scheduler can estimate future bursts from earlier measurements. Here, the user supplies each burst in advance, so the simulator can use the exact value.

5. Can SJF or SRTF cause starvation?

Yes. A continuing stream of short jobs can repeatedly push a long job out of the SJF selection or interrupt it under SRTF. The assignment workloads are finite, so arrivals eventually stop. With finite bursts and no avoidable idle time, every process eventually finishes, even if a long process waits much longer than the others.

6. Implementation mistake

No actual scheduling defect has been recorded during development so far.

AI assistance

OpenAI Codex was used to explain the scheduling rules, work through examples, help implement the simulator and edit the report. The generated code and calculations were checked against hand-calculated schedules, the Node.js tests and browser checks.